BÁO CÁO KẾT QUẢ BÀI THỰC HÀNH 7

KIỂM THỬ MOCKING ỨNG DỤNG DI ĐỘNG VÀ TÍCH HỢP LIÊN TỤC (CI/CD)

Họ và tên:	Nguyễn Hoài Nam	Mã số sinh viên:	**2312694**
Email:	2312694@dlu.edu.vn	Lớp:	CTK47 - KTPM
Kho lưu trữ GitHub:	https://github.com/namnh-2312694/yummyapp-cicd	Ngày hoàn thành:	16/10/2026
Trạng thái CI Pipeline:	□ PASSED (100% Quality Gate)	Độ bao phủ (Coverage):	99% (Vượt ngưỡng 80%)

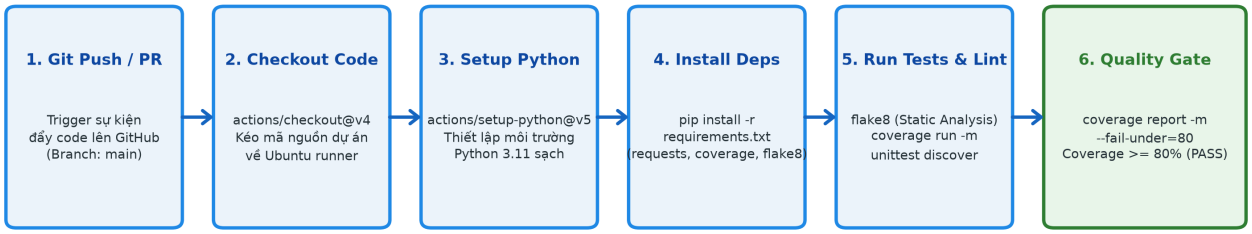
I. BỐI CẢNH DỰ ÁN & MỤC TIÊU THỰC HÀNH

Dự án YummyApp Mobile Backend là hệ thống xử lý nghiệp vụ đơn hàng ẩm thực trên thiết bị di động. Các chức năng trọng tâm bao gồm: (1) Tính toán tổng giá trị đơn hàng theo giỏ hàng; (2) Xác thực và áp dụng mã khuyến mãi theo điều kiện mạng và phiên bản hệ điều hành; (3) Đề xuất danh sách nhà hàng lân cận thông qua việc truy xuất định vị phần cứng GPS của thiết bị di động. Mục tiêu bài thực hành là áp dụng kỹ thuật Mocking (unittest.mock.patch) để cô lập phần cứng GPS và mạng viễn thông, đồng thời xây dựng đường ống CI tự động với GitHub Actions, thiết lập Cổng chất lượng (Quality Gate) nghiêm ngặt với Linter (flake8) và độ bao phủ mã nguồn (Coverage ≥ 80%).

II. SƠ ĐỒ KIẾN TRÚC ĐƯỜNG ỐNG CI/CD (CI PIPELINE)

Đường ống CI được cấu hình tự động kích hoạt mỗi khi có sự kiện push hoặc pull_request vào nhánh main. Toàn bộ quy trình diễn ra trên máy chủ sạch ubuntu-latest gồm 6 khối tuần tự:

SƠ ĐỒ KIẾN TRÚC ĐƯỜNG ỐNG CI/CD - YUMMYAPP MOBILE BACKEND



III. THIẾT KẾ MA TRẬN KỊCH BẢN KIỂM THỬ (TEST MATRIX)

Để đảm bảo độ bao phủ vượt ngưỡng Quality Gate ≥ 80%, bộ kiểm thử được thiết kế hoàn chỉnh theo 3 cấp độ:

Cấp độ / Tên Test Case	Hàm được gọi	Đối tượng & Giá trị Mocking	Kết quả kỳ vọng (Assert)

Level 1 (Easy): test_calculate_total_for_single_item test_calculate_total_for_multiple_items test_calculate_total_for_empty_cart test_calculate_total_with_zero_quantity	calculate_order_total(cart)	Không áp dụng Mocking (Dữ liệu giỏ hàng thông thường)	Tổng tiền đơn hàng chính xác theo giá thực đơn (100k, 95k, 0k)
Level 2 (Medium - Case 1): test_apply_promo_online_valid	apply_promo_code(total, 'GIAM10')	network_status = 'online', os_version = 15	Giảm 10% đơn hàng: 100,000 -> 90,000.0
Level 2 (Medium - Case 2): test_apply_promo_offline_raises_connection_error	apply_promo_code(total, 'GIAM10')	network_status = 'offline'	Ném ConnectionError: 'Khong co ket noi mang de xac thuc ma.'
Level 2 (Medium - Case 3): test_apply_promo_special_code_valid_os15	apply_promo_code(total, 'NEW_OS_ONLY')	network_status = 'online', os_version = 15	Giảm 20% đơn hàng: 200,000 -> 160,000.0
Level 2 (Medium - Case 4): test_apply_promo_special_code_lower_os14_raises_value_error	apply_promo_code(total, 'NEW_OS_ONLY')	network_status = 'online', os_version = 14	Ném ValueError: 'Ma nay yeu cau HDH phien ban 15 tro len.'
Level 3 (Hard - Case 1): test_get_nearby_restaurants_in_range	get_nearby_restaurants()	@patch('yummy_app.gps_service.get_current_location') return_value = (10.0, 105.0)	Trả về: ['Nha hang Pho Ly Quoc Su', 'Bun cha Huong Lien']
Level 3 (Hard - Case 2): test_get_nearby_restaurants_out_of_range	get_nearby_restaurants()	@patch('yummy_app.gps_service.get_current_location') return_value = (20.0, 110.0)	Trả về danh sách rỗng: []
Level 3 (Hard - Case 3): test_get_nearby_restaurants_gps_signal_lost	get_nearby_restaurants()	@patch('yummy_app.gps_service.get_current_location') side_effect = GPSSignalLostError	Bắt ngoại lệ, trả về thông báo: 'Khong the xac dinh vi tri.'

IV. KẾT QUẢ CHẠY TEST CỤC BỘ & BÁO CÁO COVERAGE

Chạy bộ kiểm thử toàn diện trên máy cục bộ bằng lệnh coverage run -m unittest discover -s . -p 'test_*.py' và xuất báo cáo chi tiết coverage report -m:

Tên tập tin mã nguồn (File)	Số câu lệnh (Stmts)	Bỏ lỡ (Miss)	Độ bao phủ (Cover)	Dòng chưa test (Missing)
gps_service.py	4	0	**100%**	-
yummy_app.py	30	0	**100%**	-
test_yummy_app.py	71	1	**99%**	Dòng 112 (main)
TỔNG CỘNG (TOTAL)	**105**	**1**	**99%**	**Vượt Quality Gate ≥ 80%**

V. MINH CHỨNG THỰC THI TRÊN GITHUB ACTIONS & VƯỢT QUALITY GATE

Kết quả đường ống CI trên GitHub Actions chạy thành công 100% (Tích xanh), toàn bộ 13 test cases đều PASSED, Linter flake8 đạt 0 lỗi, Coverage đạt 99% và đã xuất thành công Artifact báo cáo HTML:



VI. GIẢI ĐÁP CÁC CÂU HỎI LÝ THUYẾT & PHÂN TÍCH CHUYÊN SÂU

1. Cấp độ 2 - Câu 2.1: Tại sao bước chạy Linter (flake8) phải đặt trước bước chạy test trong đường ống CI?

Trả lời:
• Áp dụng nguyên lý Thất bại sớm (Fail-Fast): Linter thực hiện phân tích mã tĩnh (Static Code Analysis) để rà quét các lỗi cú pháp nghiêm trọng (Syntax Errors, E9), lỗi chưa định nghĩa biến/hàm (Undefined Names, F82), hoặc vi phạm chuẩn mã nguồn. Quá trình này diễn ra chỉ trong vài giây và tốn cực kỳ ít tài nguyên máy chủ.
• Tối ưu chi phí và thời gian thực thi CI: Quá trình chạy bộ kiểm thử Unit Test đòi hỏi nạp toàn bộ runtime, khởi tạo mock objects, đo đạc độ bao phủ mã nguồn tốn kém thời gian hơn nhiều. Nếu mã nguồn đã có lỗi cú pháp hoặc lỗi biến chưa định nghĩa, bộ test chắc chắn sẽ gãy (crash). Đặt Linter chạy trước giúp 'chặn sớm' (Break Early), hủy tiến trình ngay lập tức và giải phóng hàng đợi runner của GitHub Actions.
• Đảm bảo tính trong sạch của mã nguồn: Ngăn chặn việc lãng phí thời gian đo lường coverage trên một codebase chưa đạt tiêu chuẩn kỹ thuật hình thức tối thiểu.

2. Cấp độ 2 - Câu 2.2: Tham số --fail-under=80 của lệnh coverage report hoạt động theo nguyên lý nào trong hệ điều hành Linux để báo hiệu cho GitHub Actions dừng tiến trình build?

Trả lời:
• Nguyên lý Mã thoát (Exit Code / Return Code) trong POSIX/Linux: Trong hệ điều hành Linux, mỗi tiến trình khi kết thúc đều trả về cho tiến trình cha một mã số nguyên từ 0 đến 255. Theo quy ước chuẩn POSIX: Exit Code = 0 biểu thị lệnh thực thi thành công mỹ mãn; trong khi bất kỳ Exit Code != 0 (như 1, 2, ...) đều báo hiệu tiến trình gặp lỗi hoặc điều kiện thất bại.
• Cơ chế hoạt động của tham số --fail-under=80: Công cụ coverage sau khi tổng hợp sẽ so sánh tỷ lệ bao phủ thực tế với ngưỡng 80%. Nếu độ bao phủ < 80%, lệnh chủ động trả về Exit Code khác 0 (thường là 2).
• Cơ chế bắt lỗi của GitHub Actions Runner: Runner thực thi shell script với cờ bash -e (errexit). Khi một dòng lệnh trả về mã thoát khác 0, shell sẽ lập tức ngắt tiến trình (terminate immediately), đánh dấu bước đó là FAILED (đỏ), khóa cổng không cho chuyển sang các bước kế tiếp và ngăn chặn việc merge mã nguồn chưa đạt chuẩn.

3. Cấp độ 2 - Câu 2.3: Tham số -m trong lệnh coverage report -m hiển thị thêm thông tin gì và giúp ích gì cho kỹ sư kiểm thử khi tối ưu độ bao phủ mã nguồn?

Trả lời:
• Thông tin hiển thị thêm: Cờ -m (viết tắt của --show-missing) bổ sung thêm cột Missing trong bảng kết quả. Cột này liệt kê chi tiết số thứ tự dòng hoặc khoảng dòng lệnh cụ thể trong file chưa từng được kích hoạt (unexecuted lines) trong toàn bộ quá trình chạy kiểm thử.
• Lợi ích đối với kỹ sư kiểm thử (SET):
- Định vị chính xác điểm mù kiểm thử (Coverage Gaps): Kỹ sư không phải đọc lại toàn bộ code để phỏng đoán, mà biết ngay dòng code nào thuộc nhánh rẽ if/else hay khối ngoại lệ except chưa được duyệt.
- Viết kịch bản test case bổ sung có mục tiêu: Từ danh sách dòng bị bỏ sót, kỹ sư dễ dàng thiết kế dữ liệu đầu vào hoặc mock trạng thái thích hợp để kích hoạt đúng dòng mã đó.
- Tiết kiệm thời gian và tối ưu hóa nỗ lực: Đạt ngưỡng cổng chất lượng nhanh nhất mà không phải viết các test case dư thừa.

4. Cấp độ 3 - Câu 3.1: Phân biệt giữa khái niệm Flaky Test và lỗi do sai lệch môi trường phụ thuộc.

Trả lời:
• Khái niệm Flaky Test (Kiểm thử bất định / chập chờn):
- Là hiện tượng một test case khi chạy lặp lại nhiều lần trên cùng một phiên bản mã nguồn và trong cùng một cấu hình môi trường lại cho kết quả không nhất quán: lúc Pass, lúc Fail một cách ngẫu nhiên.
- Nguyên nhân cốt lõi: Do phụ thuộc vào các yếu tố ngoại cảnh ngoài tầm kiểm soát: xung đột bất đồng bộ (Race Conditions), thời gian chờ (Timeout/Sleep không chuẩn), sự chia sẻ trạng thái dùng chung (Shared Mutable State giữa các test), hoặc phụ thuộc trực tiếp vào kết nối mạng, sóng viễn thông, tín hiệu GPS chập chờn của thiết bị thật.
• Khái niệm Lỗi do sai lệch môi trường phụ thuộc (Environment/Dependency Drift):
- Là lỗi phát sinh khi kiểm thử chạy hoàn toàn bình thường ở máy lập trình viên (Local machine: 'It works on my

machine') nhưng lại thất bại nhất quán 100% khi chạy trên máy chủ CI/CD hoặc máy đồng nghiệp.
- Nguyên nhân cốt lõi: Do sự không đồng nhất về cấu hình hệ thống: khác biệt phiên bản hệ điều hành (Windows vs Ubuntu), khác biệt phiên bản Python (3.10 vs 3.12), không ghim cứng phiên bản thư viện dẫn đến CI tải phiên bản mới chứa Breaking Changes, hoặc khác biệt về đường dẫn file, biến môi trường, múi giờ.
• Tóm tắt phân biệt: Flaky Test mang tính bất định, thất bại ngẫu nhiên (non-deterministic); còn sai lệch môi trường là lỗi xác định, thất bại nhất quán (deterministic) trên môi trường bị sai lệch.

5. Cấp độ 3 - Câu 3.2: Tại sao trên máy chủ CI bắt buộc phải tuân thủ nguyên tắc ghim phiên bản (Version Pinning) trong requirements.txt?

Trả lời:
• Đảm bảo tính tái lập (Reproducibility) và tính xác định (Determinism): Ghim phiên bản chuẩn (ví dụ coverage==7.4.0) đảm bảo rằng mọi lượt chạy CI dù là hôm nay hay nhiều năm sau đều chạy trên cùng một tập hợp phụ thuộc chính xác, giúp kết quả build phản ánh đúng chất lượng mã nguồn dự án.
• Ngăn ngừa phá vỡ mã do cập nhật ngầm (Breaking Changes): Nếu không ghim phiên bản, pip sẽ tự động cài phiên bản mới nhất (latest). Khi một thư viện bên ngoài phát hành phiên bản mới có thay đổi API, làm mất tính tương thích ngược hoặc loại bỏ một hàm cũ, đường ống CI sẽ lập tức bị gãy (Break the build) dù nhóm dự án không sửa đổi gì trong mã nguồn.
• Bảo vệ an ninh chuỗi cung ứng phần mềm: Giúp nhóm kỹ sư kiểm soát chặt chẽ các gói cài đặt, tránh nguy cơ CI vô tình kéo về các phiên bản phát hành mới bị nhiễm mã độc (Dependency Confusion, Typosquatting).

6. Cấp độ 3 - Câu 3.3: Trình bày quy trình 6 bước chuẩn để nâng cấp một thư viện lên phiên bản mới an toàn qua Pull Request mà không làm gián đoạn nhánh main.

Trả lời - Quy trình 6 bước chuẩn mực của Kỹ sư Kiểm thử:
• Bước 1 - Tạo nhánh phụ độc lập (Feature/Chore Branch): Từ nhánh main ổn định mới nhất, tạo một nhánh riêng (ví dụ chore/upgrade-requests-to-2.32). Tuyệt đối không thử nghiệm nâng cấp trực tiếp trên main.
• Bước 2 - Đọc kỹ Release Notes & Changelog: Nghiên cứu tài liệu của phiên bản mới để nắm bắt các Breaking Changes, các hàm bị deprecate, thay đổi kiểu dữ liệu trả về và yêu cầu về runtime.
• Bước 3 - Cập nhật số phiên bản và kiểm thử cục bộ (Local Testing): Sửa số phiên bản trong requirements.txt, chạy pip install, sửa mã nguồn nếu có sự thay đổi API, và chạy toàn bộ bộ Unit Test, Linter, Coverage trên máy cá nhân để đảm bảo mọi thứ vượt qua kiểm định.
• Bước 4 - Tạo Pull Request (PR) & Kích hoạt CI Pipeline tự động: Commit thay đổi, đẩy lên kho lưu trữ và tạo Pull Request vào nhánh main. GitHub Actions sẽ tự động dựng môi trường sạch, chạy Linter, Unit Test và Quality Gate độc lập.
• Bước 5 - Đánh giá mã nguồn (Code Review) & Kiểm duyệt chất lượng: Đồng nghiệp xem xét các thay đổi diff, kiểm tra kết quả CI đảm bảo 100% Tích xanh (Pass) và không có nguy cơ hồi quy (Regression).
• Bước 6 - Hợp nhất an toàn (Merge PR) & Giám sát sau triển khai: Thực hiện Merge PR vào nhánh main. Đường ống CI trên nhánh main tự động chạy lại để xác nhận trạng thái tích hợp hoàn hảo.

Đà Lạt, ngày 16 tháng 10 năm 2026

Sinh viên thực hiện

Nam

Nguyễn Hoài Nam - 2312694